

Real-time Infrastructure – Django Channels & WebSocket

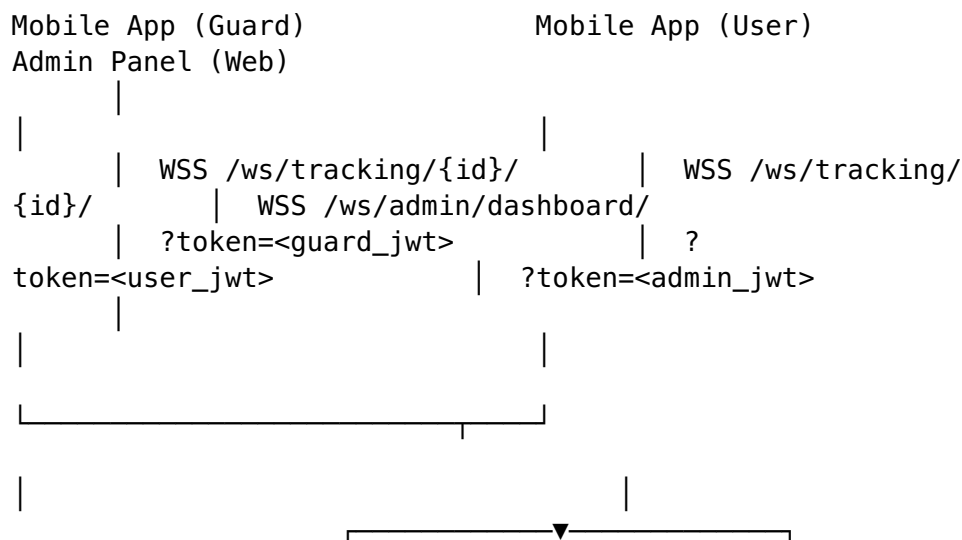
Tech: Django Channels 4.x, Redis Channel Layer, Daphne ASGI server

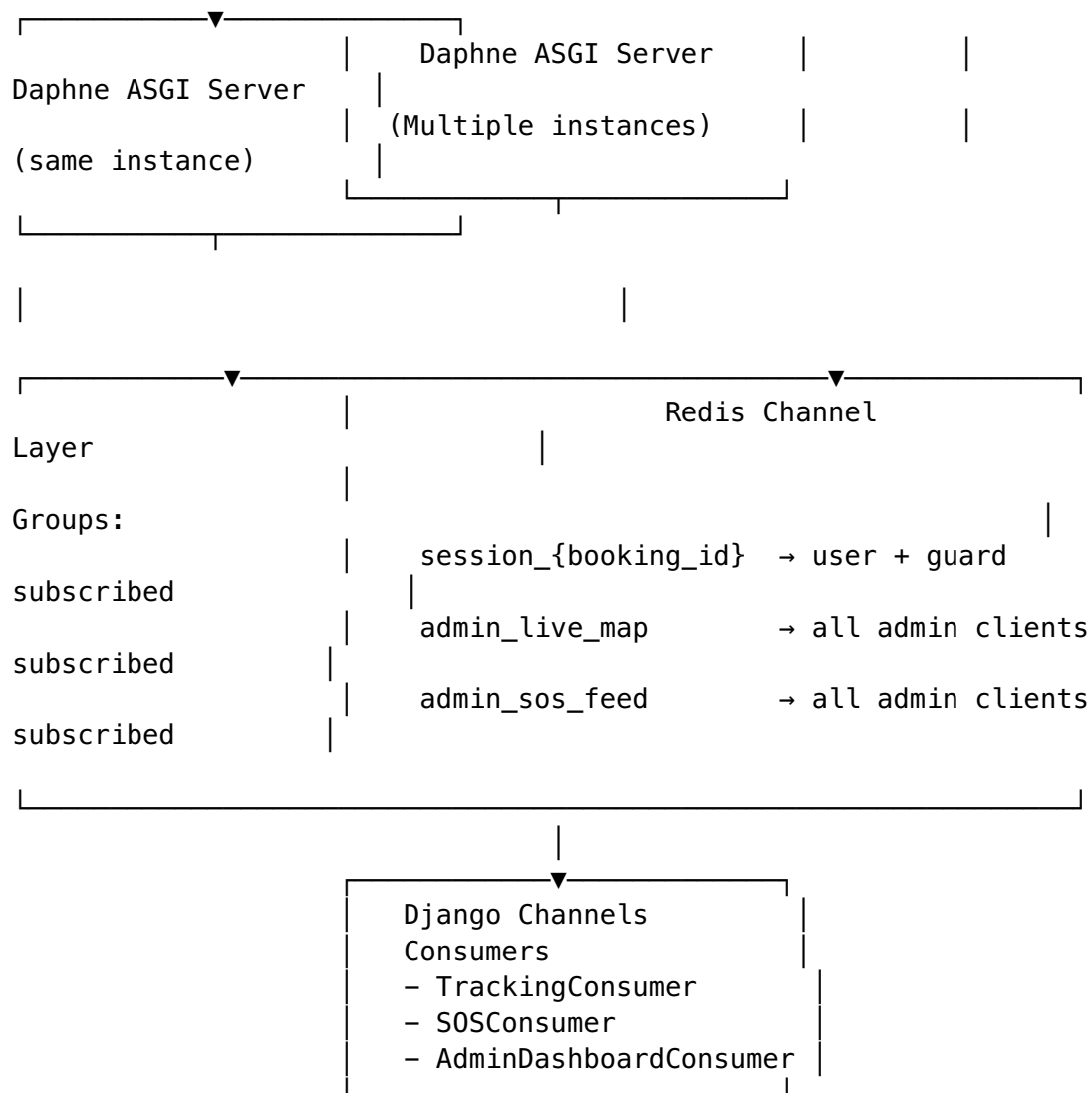
Protocol: WebSocket (WSS in production)

Table of Contents

1. [Architecture Overview](#)
 2. [ASGI Configuration](#)
 3. [WebSocket Authentication Middleware](#)
 4. [TrackingConsumer – Live Guard Location](#)
 5. [SOSConsumer – Admin Real-time Alerts](#)
 6. [AdminDashboardConsumer](#)
 7. [WebSocket URL Routing](#)
 8. [Redis Channel Layer Configuration](#)
 9. [Client Integration Guide](#)
 10. [Message Protocol Reference](#)
 11. [Error Handling & Reconnection](#)
 12. [Scaling WebSocket Connections](#)
-

1. Architecture Overview





Key design decisions:

- **Redis as channel layer** — all Daphne instances share one Redis. A message published by one instance is received by all others. This enables horizontal scaling.
- **Group-based messaging** — no point-to-point connections. Guard broadcasts location to group `session_{id}`; both user and admin (if watching) receive it.
- **JWT in query param** — the only viable option for WebSocket auth (no custom headers in WS handshake).
- **Guard is the only writer** for location updates; user and admin are read-only in tracking groups.

2. ASGI Configuration

```
# config/asgi.py
```

```
import os
import django
from django.core.asgi import get_asgi_application
```

```

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
                      'config.settings.production')
django.setup()

from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import
    AllowedHostsOriginValidator

# Import after django.setup() to avoid AppRegistryNotReady
from utils.ws_middleware import JWTAuthMiddlewareStack
from apps.tracking.routing import tracking_ws_urlpatterns
from apps.sos.routing import sos_ws_urlpatterns
from apps.admin_panel.routing import admin_ws_urlpatterns

application = ProtocolTypeRouter({
    # All HTTP requests go through normal Django views
    'http': get_asgi_application(),

    # All WebSocket requests go through Channels routing
    'websocket': AllowedHostsOriginValidator(
        JWTAuthMiddlewareStack(
            URLRouter(
                tracking_ws_urlpatterns +
                sos_ws_urlpatterns +
                admin_ws_urlpatterns
            )
        )
    ),
})

```

3. WebSocket Authentication Middleware

```

# utils/ws_middleware.py

from urllib.parse import parse_qs
from channels.middleware import BaseMiddleware
from channels.db import database_sync_to_async
from django.contrib.auth.models import AnonymousUser
from rest_framework_simplejwt.tokens import AccessToken
from rest_framework_simplejwt.exceptions import TokenError,
    InvalidToken
import logging

logger = logging.getLogger(__name__)

@database_sync_to_async
def get_user_from_jwt(token_str: str):
    """Validate JWT and return associated UserProfile, or
    AnonymousUser on failure."""
    from apps.users.models import UserProfile

```

```

try:
    token = AccessToken(token_str)
    user_id = token.get('user_id')
    if not user_id:
        return AnonymousUser()
    return
    UserProfile.objects.select_related('guard_profile').get(id=user_id)
except (TokenError, InvalidToken):
    return AnonymousUser()
except UserProfile.DoesNotExist:
    return AnonymousUser()
except Exception as e:
    logger.warning(f'WS auth error: {e}')
    return AnonymousUser()

class JWTAuthMiddleware(BaseMiddleware):
    """
    Extracts JWT token from WebSocket URL query string.
    Sets scope['user'] for use in consumers.

    Example WS URL: wss://api.bsecure.in/ws/tracking/uuid/?
        token=eyJhbG...
    """

    async def __call__(self, scope, receive, send):
        query_string = scope.get('query_string',
            b'').decode('utf-8')
        query_params = parse_qs(query_string)
        token_list = query_params.get('token', [None])
        token_str = token_list[0] if token_list else None

        if token_str:
            scope['user'] = await get_user_from_jwt(token_str)
        else:
            scope['user'] = AnonymousUser()

        return await super().__call__(scope, receive, send)

def JWTAuthMiddlewareStack(inner):
    """Wrap inner application with JWT authentication."""
    return JWTAuthMiddleware(inner)

```

4. TrackingConsumer — Live Guard Location

apps/tracking/consumers.py

```

import json
import logging
from channels.generic.websocket import AsyncWebsocketConsumer

```

```
from channels.db import database_sync_to_async
from django.utils import timezone
from django.contrib.gis.geos import Point
```

```
logger = logging.getLogger(__name__)
```

```
class TrackingConsumer(AsyncWebsocketConsumer):
    """
    WebSocket consumer for real-time guard location tracking.

    URL: /ws/tracking/{booking_id}/

    Roles:
    - Guard: sends location updates, receives session status
      changes
    - User: receives location updates and session status changes
    - Admin: receives location updates (joined to admin_live_map
      group separately)

    Groups:
    - session_{booking_id}: all participants of this booking
    - admin_live_map: all admin clients watching the live map
    """
```

```
async def connect(self):
    self.booking_id = self.scope['url_route']['kwargs']
        ['booking_id']
    self.session_group = f'session_{self.booking_id}'
    self.user = self.scope['user']

    # Reject unauthenticated connections
    if self.user.is_anonymous:
        await self.close(code=4001)
        return

    # Verify user is participant in this booking
    booking = await self._get_booking()
    if not booking:
        await self.close(code=4004)
        return

    if not await self._is_participant(booking):
        await self.close(code=4003)
        return

    self.booking = booking
    self.is_guard = await self._check_is_guard()

    # Join session group
    await self.channel_layer.group_add(self.session_group,
        self.channel_name)
```

```

# If admin, also join admin live map group
if self.user.is_staff:
    await self.channel_layer.group_add('admin_live_map',
self.channel_name)

await self.accept()
logger.info(f'WS connected: user={self.user.id}
booking={self.booking_id} guard={self.is_guard}')

# Send current booking status on connect
await self.send(text_data=json.dumps({
    'type': 'session_state',
    'status': self.booking.status,
    'booking_id': str(self.booking_id),
}))

async def disconnect(self, close_code):
    if hasattr(self, 'session_group'):
        await
self.channel_layer.group_discard(self.session_group,
self.channel_name)
    if self.user.is_staff:
        await
self.channel_layer.group_discard('admin_live_map',
self.channel_name)
    logger.info(f'WS disconnected: user={self.user.id}
code={close_code}')

async def receive(self, text_data):
    """
    Only guards send messages. Users and admins are read-only.
    """
    if not self.is_guard:
        # Non-guards cannot send messages on this channel
        await self.send(text_data=json.dumps({
            'type': 'error',
            'code': 'NOT_PERMITTED',
            'message': 'Only the assigned guard can send
location updates.'
        }))
        return

    try:
        data = json.loads(text_data)
    except json.JSONDecodeError:
        return

    message_type = data.get('type')

    if message_type == 'location_update':
        await self._handle_location_update(data)
    elif message_type == 'ping':
        await self.send(text_data=json.dumps({'type':
'pong'}))

```

```

async def _handle_location_update(self, data: dict):
    """
    Process guard location update:
    1. Save to DB (async)
    2. Broadcast to session group (user + admin)
    3. Broadcast to admin live map group
    """

    lat = data.get('lat')
    lng = data.get('lng')
    accuracy = data.get('accuracy')

    if not lat or not lng:
        return

    # Save snapshot to DB
    await self._save_location_snapshot(lat, lng, accuracy,
    data.get('speed'), data.get('bearing'))

    # Update guard's current location on GuardProfile
    await self._update_guard_location(lat, lng)

    # Compute ETA (async call to Google Maps Distance Matrix)
    eta_seconds = await self._get_eta(lat, lng)

    # Broadcast to all in session group
    outbound = {
        'type': 'guard_location',
        'lat': lat,
        'lng': lng,
        'accuracy': accuracy,
        'speed': data.get('speed'),
        'bearing': data.get('bearing'),
        'eta_seconds': eta_seconds,
        'timestamp': timezone.now().isoformat(),
    }
    await self.channel_layer.group_send(self.session_group, {
        'type': 'broadcast_message',
        'payload': outbound,
    })

    # Also broadcast to admin live map (with booking context)
    await self.channel_layer.group_send('admin_live_map', {
        'type': 'broadcast_message',
        'payload': {
            **outbound,
            'booking_id': str(self.booking_id),
            'guard_id': str(self.user.guard_profile.id),
        }
    })

# --- Channel layer message handlers (called when group sends
# to this consumer) ---

```

```

async def broadcast_message(self, event):
    """Forward any group message to the WebSocket client."""
    await self.send(text_data=json.dumps(event['payload']))

```

```

async def session_status_update(self, event):
    """Called when booking status changes – notify all connected clients."""
    await self.send(text_data=json.dumps({
        'type': 'session_status_change',
        'status': event['status'],
        'timestamp': event.get('timestamp'),
    }))

```

```

# --- Database helpers (must be async) ---

```

```

@database_sync_to_async
def _get_booking(self):
    from apps.bookings.models import Booking
    try:
        return Booking.objects.select_related('user',
            'guard__user').get(
                id=self.booking_id
            )
    except Booking.DoesNotExist:
        return None

```

```

@database_sync_to_async
def _is_participant(self, booking) -> bool:
    if self.user.is_staff:
        return True
    if booking.user_id == self.user.id:
        return True
    if hasattr(self.user, 'guard_profile') and booking.guard
    == self.user.guard_profile:
        return True
    return False

```

```

@database_sync_to_async
def _check_is_guard(self) -> bool:
    return (
        hasattr(self.user, 'guard_profile') and
        self.booking.guard == self.user.guard_profile
    )

```

```

@database_sync_to_async
def _save_location_snapshot(self, lat, lng, accuracy, speed,
    bearing):
    from apps.tracking.models import LocationSnapshot
    LocationSnapshot.objects.create(
        booking=self.booking,
        guard=self.booking.guard,
        location=Point(lng, lat, srid=4326),
    )

```

```

        accuracy_meters=accuracy,
        speed_kmh=speed,
        bearing_degrees=bearing,
        timestamp=timezone.now(),
    )

@database_sync_to_async
def _update_guard_location(self, lat, lng):
    from apps.guards.models import GuardProfile
    GuardProfile.objects.filter(id=self.booking.guard_id).update(
        current_location=Point(lng, lat, srid=4326),
        last_location_update=timezone.now(),
    )

async def _get_eta(self, guard_lat, guard_lng) -> int | None:
    """
    Calls Google Maps Distance Matrix API to get ETA.
    Only during EN_ROUTE phase (guard travelling to user).
    Returns ETA in seconds, or None if not applicable.
    """
    if self.booking.status != 'EN_ROUTE':
        return None

    # Import here to avoid circular imports
    from utils.maps import get_driving_duration_seconds
    try:
        return await get_driving_duration_seconds(
            origin=(guard_lat, guard_lng),
            destination=(
                float(self.booking.service_latitude),
                float(self.booking.service_longitude)
            )
        )
    except Exception:
        return None

```

Triggering Status Updates from Django Views

When booking status changes in a view/service, broadcast to all connected WS clients:

```

# apps/bookings/services.py

from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from django.utils import timezone

class BookingService:

    @staticmethod
    def broadcast_status_change(booking):

```

```

"""
Broadcast booking status change to all WebSocket clients
in the session group.
Called after any booking state transition.
"""

channel_layer = get_channel_layer()
group_name = f'session_{booking.id}'

async_to_sync(channel_layer.group_send)(group_name, {
    'type': 'session_status_update',
    'status': booking.status,
    'timestamp': timezone.now().isoformat(),
})

```

5. SOSConsumer — Admin Real-time Alerts

apps/sos/consumers.py

```

import json
from channels.generic.websocket import AsyncWebsocketConsumer

class SOSFeedConsumer(AsyncWebsocketConsumer):
    """
    Admin-only WebSocket for real-time SOS alert feed.
    URL: /ws/sos/feed/

    All admin clients subscribe to group 'admin_sos_feed'.
    When an SOS is triggered (via REST API), it's broadcast here.
    """

    async def connect(self):
        user = self.scope['user']

        if user.is_anonymous or not user.is_staff:
            await self.close(code=4001)
            return

        await self.channel_layer.group_add('admin_sos_feed',
            self.channel_name)
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard('admin_sos_feed',
            self.channel_name)

    async def receive(self, text_data):
        # Admin can acknowledge SOS via WebSocket
        data = json.loads(text_data)
        if data.get('type') == 'acknowledge_sos':
            await self._handle_acknowledge(data)

```

```

async def sos_alert(self, event):
    """Called when a new SOS is triggered anywhere on the
    platform."""
    await self.send(text_data=json.dumps(event['payload']))

async def sos_status_update(self, event):
    """Called when SOS status changes (acknowledged,
    resolved)."""
    await self.send(text_data=json.dumps(event['payload']))

async def _handle_acknowledge(self, data):
    from channels.db import database_sync_to_async
    sos_id = data.get('sos_id')
    # Update in DB
    await database_sync_to_async(self._ack_sos)(sos_id)

def _ack_sos(self, sos_id):
    from apps.sos.models import SOSAlert
    from django.utils import timezone
    SOSAlert.objects.filter(id=sos_id,
        status='TRIGGERED').update(
        status='ACKNOWLEDGED',
        acknowledged_at=timezone.now(),
        assigned_to=self.scope['user'],
    )

```

Broadcasting SOS from REST API

```

# apps/sos/services.py

from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer

class SOSService:

    @staticmethod
    def trigger_sos(user, booking, trigger_method, latitude,
        longitude) -> 'SOSAlert':
        from .models import SOSAlert
        from django.utils import timezone

        # Step 1: Save SOS record synchronously (MUST be instant)
        sos = SOSAlert.objects.create(
            user=user,
            booking=booking,
            trigger_method=trigger_method,
            latitude=latitude,
            longitude=longitude,
            status='TRIGGERED',
        )

```

```

# Step 2: Broadcast to all admin SOS watchers via
WebSocket
SOSService._broadcast_to_admins(sos)

# Step 3: Async – notify emergency contacts + guard
from .tasks import notify_emergency_contacts,
notify_guard_of_user_sos
notify_emergency_contacts.apply_async(args=[str(sos.id)],
queue='high_priority')
if booking:

    notify_guard_of_user_sos.apply_async(args=[str(booking.id)],
queue='high_priority')

return sos

@staticmethod
def _broadcast_to_admins(sos):
    channel_layer = get_channel_layer()
    payload = {
        'type': 'sos_alert',
        'payload': {
            'event': 'NEW_SOS',
            'sos_id': str(sos.id),
            'user_id': str(sos.user_id),
            'user_name': sos.user.display_name,
            'booking_id': str(sos.booking_id) if
sos.booking_id else None,
            'trigger_method': sos.trigger_method,
            'latitude': float(sos.latitude),
            'longitude': float(sos.longitude),
            'triggered_at': sos.created_at.isoformat(),
        }
    }
    async_to_sync(channel_layer.group_send)('admin_sos_feed',
payload)

```

6. AdminDashboardConsumer

```

# apps/admin_panel/consumers.py

import json
from channels.generic.websocket import AsyncWebsocketConsumer

class AdminDashboardConsumer(AsyncWebsocketConsumer):
    """
    Admin live dashboard WebSocket.
    URL: /ws/admin/dashboard/

    Receives:

```

- Live KPI updates (every 30 seconds, pushed by Celery beat)
- New booking notifications
- Guard online/offline events

```
"""
```

```

async def connect(self):
    user = self.scope['user']
    if user.is_anonymous or not user.is_staff:
        await self.close(code=4001)
        return

    await self.channel_layer.group_add('admin_dashboard',
    self.channel_name)
    await self.accept()

    # Send initial stats on connect
    stats = await self._get_live_stats()
    await self.send(text_data=json.dumps({
        'type': 'initial_stats',
        'data': stats
    })))

async def disconnect(self, close_code):
    await self.channel_layer.group_discard('admin_dashboard',
    self.channel_name)

async def dashboard_update(self, event):
    await self.send(text_data=json.dumps(event['payload']))

@staticmethod
def broadcast_stats_update(stats: dict):
    """Called by Celery beat task every 30 seconds."""
    from asgiref.sync import async_to_sync
    from channels.layers import get_channel_layer
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)
    ('admin_dashboard', {
        'type': 'dashboard_update',
        'payload': {
            'event': 'STATS_UPDATE',
            'data': stats
        }
    })

from channels.db import database_sync_to_async

@database_sync_to_async
def _get_live_stats(self):
    from apps.bookings.models import Booking
    from apps.guards.models import GuardProfile
    from apps.sos.models import SOSAlert
    return {

```

```
        'active_sessions':
            Booking.objects.filter(status='ACTIVE').count(),
        'guards_online':
            GuardProfile.objects.filter(is_online=True).count(),
        'open_sos_alerts': SOSAlert.objects.exclude(
            status__in=['RESOLVED', 'FALSE_ALARM']
        ).count(),
    }
}
```

7. WebSocket URL Routing

```
# apps/tracking/routing.py
from django.urls import re_path
from .consumers import TrackingConsumer

tracking_ws_urlpatterns = [
    re_path(
        r'^ws/tracking/(?P<booking_id>[0-9a-f-]{36})/$',
        TrackingConsumer.as_asgi()
    ),
]

# apps/sos/routing.py
from django.urls import re_path
from .consumers import SOSFeedConsumer

sos_ws_urlpatterns = [
    re_path(r'^ws/sos/feed/$', SOSFeedConsumer.as_asgi()),
]

# apps/admin_panel/routing.py
from django.urls import re_path
from .consumers import AdminDashboardConsumer

admin_ws_urlpatterns = [
    re_path(r'^ws/admin/dashboard/$',
        AdminDashboardConsumer.as_asgi()),
]
```

8. Redis Channel Layer Configuration

```
# config/settings/base.py

CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
```

```

        'hosts': [env('REDIS_URL')], # e.g. redis://
        redis:6379/1
        'capacity': 1500,
        # Max messages in flight per channel
        'expiry': 10, # Messages expire after
        10 seconds if undelivered
        'group_expiry': 86400, # Groups auto-expire
        after 24 hours
        'symmetric_encryption_keys':
        [env('CHANNEL_LAYER_ENCRYPTION_KEY', default=None)],
    },
},
}

```

Redis key patterns used by Channels:

asgi:group:session_{booking_id}	→ set of channel names in this group
asgi:group:admin_live_map	→ set of admin channel names
asgi:group:admin_sos_feed	→ set of admin channel names
asgi:channel:{channel_name}	→ message queue for a specific connection

9. Client Integration Guide

React Native (Guard App) — Sending Location

```

// GuardTrackingService.js

import { useEffect, useRef } from 'react';
import * as Location from 'expo-location';

const LOCATION_UPDATE_INTERVAL_MS = 4000; // 4 seconds

export function useGuardTracking(bookingId, accessToken) {
  const wsRef = useRef(null);
  const locationSubscriptionRef = useRef(null);

  useEffect(() => {
    if (!bookingId || !accessToken) return;

    // Connect WebSocket
    const ws = new WebSocket(
      `wss://api.bsecure.in/ws/tracking/${bookingId}?`
      `token=${accessToken}`
    );

    ws.onopen = () => {
      console.log('[WS] Tracking connected');
      startLocationTracking(ws);
    };
  });
}

```

```

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  handleIncomingMessage(message);
};

ws.onclose = (event) => {
  console.log('[WS] Disconnected, code:', event.code);
  stopLocationTracking();
  // Reconnect logic (exponential backoff)
  if (event.code !== 1000) {
    scheduleReconnect(bookingId, accessToken);
  }
};

wsRef.current = ws;

return () => {
  stopLocationTracking();
  ws.close(1000, 'Component unmounted');
};
}, [bookingId, accessToken]);

const startLocationTracking = async (ws) => {
  const { status } = await
    Location.requestBackgroundPermissionsAsync();
  if (status !== 'granted') return;

  locationSubscriptionRef.current = await
    Location.watchPositionAsync(
      {
        accuracy: Location.Accuracy.BestForNavigation,
        timeInterval: LOCATION_UPDATE_INTERVAL_MS,
        distanceInterval: 5, // Only update if moved >5m
      },
      (location) => {
        if (ws.readyState === WebSocket.OPEN) {
          ws.send(JSON.stringify({
            type: 'location_update',
            lat: location.coords.latitude,
            lng: location.coords.longitude,
            accuracy: location.coords.accuracy,
            speed: location.coords.speed * 3.6, // m/
              s → km/h
            bearing: location.coords.heading,
          }));
        }
      }
    );
};

const stopLocationTracking = () => {
  if (locationSubscriptionRef.current) {

```

```

        locationSubscriptionRef.current.remove();
      }
    };
  }
}

```

React Native (User App) — Receiving Location

// UserTrackingMap.js

```

import { useEffect, useState } from 'react';

export function useGuardLocationTracking(bookingId, accessToken) {
  const [guardLocation, setGuardLocation] = useState(null);
  const [sessionStatus, setSessionStatus] = useState(null);
  const [etaSeconds, setEtaSeconds] = useState(null);
  const wsRef = useRef(null);

  useEffect(() => {
    if (!bookingId) return;

    const ws = new WebSocket(
      `wss://api.bsecure.in/ws/tracking/${bookingId}?token=${accessToken}`
    );

    ws.onmessage = (event) => {
      const message = JSON.parse(event.data);

      switch (message.type) {
        case 'guard_location':
          setGuardLocation({ lat: message.lat, lng: message.lng });
          setEtaSeconds(message.eta_seconds);
          break;
        case 'session_status_change':
          setSessionStatus(message.status);
          break;
        case 'session_state':
          setSessionStatus(message.status);
          break;
      }
    };

    wsRef.current = ws;

    return () => ws.close(1000, 'Unmounted');
  }, [bookingId]);

  return { guardLocation, sessionStatus, etaSeconds };
}

```

10. Message Protocol Reference

Guard → Server

Message Type	When Sent	Payload
location_update	Every 4s during active session	{lat, lng, accuracy, speed, bearing}
ping	Every 30s keepalive	{}

Server → Client (broadcast to session group)

Message Type	When Sent	Payload
guard_location	On each guard location update	{lat, lng, accuracy, speed, bearing, eta_seconds, timestamp}
session_status_change	On booking state transition	{status, timestamp}
session_state	On initial WS connect	{status, booking_id}
pong	Response to ping	{}
error	On bad message	{code, message}

Server → Admin SOS Feed

Message Type	When Sent	Payload
sos_alert	New SOS triggered	{event: 'NEW_SOS', sos_id, user_name, latitude, longitude, trigger_method, triggered_at}
sos_status_update	SOS status changes	{sos_id, new_status, timestamp}

Close Codes

Code	Meaning
1000	Normal closure

Code	Meaning
4001	Authentication failed (invalid/missing token)
4003	Forbidden (not a participant in this booking)
4004	Booking not found

11. Error Handling & Reconnection

Exponential Backoff Reconnection (Client)

// utils/websocket.js

```
export function createReconnectingWebSocket(url, options = {}) {
  const {
    maxRetries = 10,
    baseDelay = 1000,
    maxDelay = 30000,
    onMessage,
    onStatusChange,
  } = options;

  let retryCount = 0;
  let ws = null;

  const connect = () => {
    ws = new WebSocket(url);

    ws.onopen = () => {
      retryCount = 0;
      onStatusChange?.('CONNECTED');
    };

    ws.onmessage = (event) => {
      onMessage?.(JSON.parse(event.data));
    };

    ws.onclose = (event) => {
      if (event.code === 1000) return; // Intentional close

      if (retryCount < maxRetries) {
        const delay = Math.min(baseDelay * Math.pow(2,
          retryCount), maxDelay);
        retryCount++;
        onStatusChange?.(`RECONNECTING (attempt $
          {retryCount})`);
        setTimeout(connect, delay);
      } else {
        onStatusChange?.('FAILED');
      }
    }
  };
}
```

```

    };
};

connect();

return {
    send: (data) => ws?.readyState === WebSocket.OPEN &&
    ws.send(JSON.stringify(data)),
    close: () => ws?.close(1000),
};
}

```

12. Scaling WebSocket Connections

Capacity Planning

- Each active session creates 2 WebSocket connections (user + guard) = 2 connections
- 10,000 concurrent sessions = ~20,000 WebSocket connections
- Single Daphne process handles ~1,000–2,000 connections comfortably
- Need **10–20 Daphne instances** for 10,000 concurrent sessions

Multi-Instance Setup

nginx.conf – WebSocket upstream pool

```

upstream daphne_pool {
    least_conn;
    server daphne1:8000;
    server daphne2:8000;
    server daphne3:8000;
    server daphne4:8000;
}

server {
    listen 443 ssl;

    location /ws/ {
        proxy_pass http://daphne_pool;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 3600;    # 1 hour – keep WS open
        proxy_send_timeout 3600;
    }
}

```

```
location /api/ {  
    proxy_pass http://daphne_pool;  
    # Standard HTTP proxy headers  
}  
}
```

Why Redis Channel Layer works for multi-instance:

Instance A (Guard connected) → publishes location to Redis group
Redis → notifies Instance B (User connected to same group)
Instance B → sends location to user's WebSocket connection

No inter-instance communication needed. Redis is the shared bus.